Università della Calabria

Dipartimento di Ingegneria Informatica, Modellistica, Elettronica e Sistemistica

---

Master Degree in ‘Robotics And Automation Engineering’

Course: Filtering and Identification of Dynamical Systems

Kalman Filtering and RTS Smoothing of a Quarter-Car System

Student

Alessandro Furina
252328

Professors

Giuseppe Fedele
Luigi D’Alfonso

Academic Year 2024-2025

TABLE OF CONTENTS

INTRODUCTION

The aim of this project consists of analysing a dynamical system in presence of noise. Noisy signals are very common in real life due to the interaction of systems with the external environment. Since each dynamical system is affected by this interaction, a theory capable of extending the main concepts related to system theory is necessary. In general, the following state-space representation will be considered in this work:

$$\begin{cases} x_{k+1} = A * x_k + B * u_k + v1_k \\ y_k = C * x_k + D * u_k + v2_k \\ x0 = x_0 \end{cases} \qquad (\boldsymbol{a})$$

$$A \in R^{nxn}, B \in R^{nxm}, C \in R^{pxn}, D \in R^{pxm}$$

The signals $v1_k \in R^n$ and $v2_k \in R^p$ represent the two main categories of noise:

- $\boldsymbol{v1_k}$ **represents the Process Noise**: it includes all the uncertainties related to the modelling phase, indeed it affects the state difference equation. It is responsible for identifying and quantifying the difference between the designed model and the real plant. In this formulation it plays as a disturbance during the evolution of the model.
- $\boldsymbol{v2_k}$ **represents the Measurement Noise:** it includes all the uncertainty related to the acquisition of the state information due to inaccuracy of the sensors. It is known that each sensor is affected by a distortion of the measurement due to the interaction with the environment (temperature, electro-magnetic fields and so on) or simply to manufacturing inaccuracy. Like $v1_k$ it is modelled as a disturbance.

In order to design a controller capable of satisfying some performance requirements, fundamental step is to obtain filtered signals in terms of state and output. This work is oriented to sort off this problem by exploiting Kalman filter theory. After, the problem of improving the results will be faced by resorting smoothing theory, in particular the RTS (Rauch-Tung-Striebel) algorithm.

1. MODELING

The modelling phase consists of designing a model which properly describes the behaviour (in terms of evolution) of the plant under investigation. By considering the system ***(a)*** , it is clear that we want to put our attention on a specified class of models that are the stochastic LTI systems. This category inherits all the properties of LTI system, but it characterized for the presence of the stochastic signals $v1_k$ and $v2_k$ defined by means of their PDF (Probability density function). In this work we will consider a Normally Distributed function for both the signals, that is to say:

$$v1_k \sim N(0, V_1)$$

$$v2_k \sim N(0, V_2)$$

$$V_1 \in R^{nxn}, V_2 \in R^{pxp}$$

With V_1 and V_2 covariance matrices of the respective vectors:

$$V_1 = \begin{bmatrix} Var(v1_k(1)) & \cdots & Cov(v1_k(1), v1_k(n)) \\ \vdots & \ddots & \vdots \\ Cov(v1_k(n), v1_k(1) & \cdots & Var(v1_k(n)) \end{bmatrix}$$

$$V_2 = \begin{bmatrix} Var(v2_k(1)) & \cdots & Cov(v2_k(1), v2_k(p)) \\ \vdots & \ddots & \vdots \\ Cov(v2_k(p), v2_k(1) & \cdots & Var(v2_k(p)) \end{bmatrix}$$

Where $v1_k(i)$, $v2_k(j)$ for i=1…n , j=1…p are the i-th and j-th components of the vectors at time k.

This means that at each time instant the process and measurement noise will assume different values according to the parameters of the Gaussian distributions. The choice of having an LTI stochastic model is motivated by the fact that Kalman theory (at least in its basic formulation) is based on this class of systems.

Now it is possible to introduce the model in analysis that is the ***Quarter-Car Model***. It represents the simplest way to describe the dynamic of a car in which only two degrees of freedom are considered: the displacement of the sprung mass and of the unsprung mass. According to this representation, all the dynamic of the vehicle is described in a quarter of it, hence the name. The following figure illustrates the model:

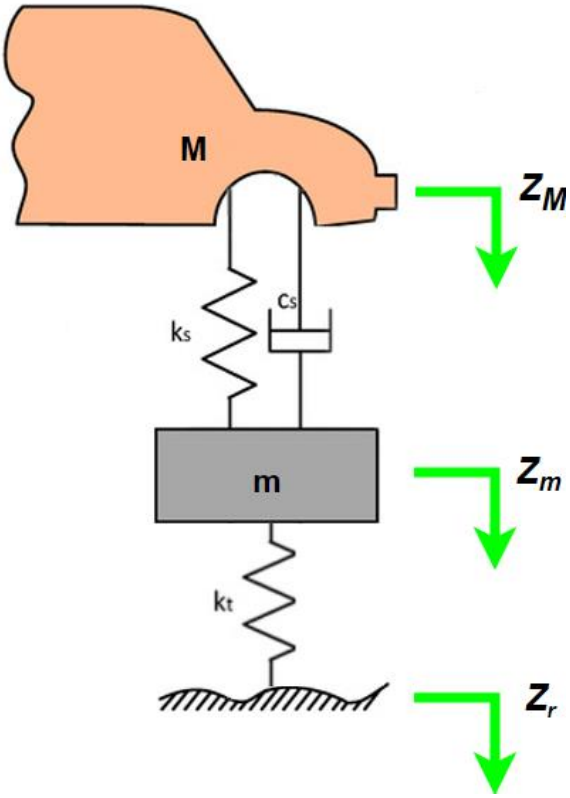

Fig. 1.1: Quarter-Car Model

Where:

- **M [250 kg]:** represents the sprung mass: it includes all the components over the suspension system such as chassis, passengers, mechanical components.
- **m [25 kg]:** represents the unsprung mass: it includes all the components under the suspension system such as axles, drive train, wheels.
- **K_s [8000 N/m]:** is the elastic coefficient of the suspension.
- **C_s [1000 N*s/m]:** is the damping coefficient of the damper.
- **K_t[160000 N/m]:** is the elastic coefficient of the wheels.
- **Z_M [cm]:** is the displacement of the sprung mass.
- **Z_m [cm]:** is the displacement of the unsprung mass.
- **Z_r[cm]:** is the road profile.

In order to obtain a model, it's necessary to analyze the equations of the free body diagram. Since the system has two degrees of freedom, we need to derive two dynamical equations: one for the sprung mass and the other for the unsprung mass. Therefore, applying the 2nd Newton law to both the masses we deal with the following equations:

Equation for M:

$$-M * \ddot{Z}_M - K_s * Z_M - C_s * \dot{Z}_M + K_s * Z_m + C_s * \dot{Z}_m = 0 \qquad (1.1)$$

Where:

- $-M * \ddot{Z}_M$ represents the inertia force (opposite to the movement)
- $-K_s * Z_M$ represents the elastic force due to the displacement of M.
- $-C_s * \dot{Z}_M$ represents the damping force due to the velocity of M.
- $K_s * Z_m$ represents the elastic force contribute due to the displacement of m.
- $C_s * \dot{Z}_m$ represents the damping force contribute due to the velocity of m.

By regrouping with respect to K_s and C_s the equation becomes:

$$-M * \ddot{Z}_M + K_s * (Z_m - Z_M) + C_s * \left(\dot{Z}_m - \dot{Z}_M\right) = 0$$

Applying the same arguments, it is possible to obtain the equation for m:

Equation for m:

$$\boldsymbol{-m * \ddot{Z}_m - K_s * Z_m - C_s * \dot{Z}_m + K_s * Z_M + C_s * \dot{Z}_M - K_t * Z_m + K_t * Z_r = 0} \qquad \textbf{(1.2)}$$

In which it appears also the contribute on K_t of the road profile.

Equation ***(1.2)*** can be rewritten as follows:

$$-m * \ddot{Z}_m + K_s * (Z_M - Z_m) + C_s * \left(\dot{Z}_M - \dot{Z}_m\right) + K_t * (Z_r - Z_m) = 0$$

So, the overall dynamics is:

$$\begin{cases} \boldsymbol{-M * \ddot{Z}_M + K_s * (Z_m - Z_M) + C_s * \left(\dot{Z}_m - \dot{Z}_M\right) = 0} \\ \boldsymbol{-m * \ddot{Z}_m + K_s * (Z_M - Z_m) + C_s * \left(\dot{Z}_M - \dot{Z}_m\right) + K_t * (Z_r - Z_m) = 0} \end{cases} \qquad \textbf{(1.3)}$$

As expected, there aren't non linearities in the above system. Next step consists of deriving a state-space representation. Hence, it is necessary to identify the state, input and output variables. Let's assign the state variables as follows:

$$\begin{bmatrix} Z_M \\ Z_m \\ \dot{Z}_M \\ \dot{Z}_m \end{bmatrix} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} \in R^4$$

So, we are measuring the displacements and the velocities of the two masses.

It is natural to select the road profile as input:

$$Z_r = u \in R$$

While for what concerns the output there are several options. If the analysis wants to investigate the stability of the vehicle, the displacement and velocity of the unsprung mass should be chosen. On the contrary, if we want to put more emphasis on the comfort of the passengers, we should consider the displacement and velocity of the sprung mass. In this work the last scenario was selected.

By replacing the state-space variables in the system ***(1.3)*** we obtain:

$$\begin{cases} -\boldsymbol{M} * \dot{\boldsymbol{x}}_{\boldsymbol{3}} - \boldsymbol{K_s} * \boldsymbol{x_1} - \boldsymbol{C_s} * \boldsymbol{x_3} + \boldsymbol{K_s} * \boldsymbol{x_2} + \boldsymbol{C_s} * \boldsymbol{x_4} = \boldsymbol{0} \\ -\boldsymbol{m} * \dot{\boldsymbol{x}}_{\boldsymbol{4}} - \boldsymbol{K_s} * \boldsymbol{x_2} - \boldsymbol{C_s} * \boldsymbol{x_4} + \boldsymbol{K_s} * \boldsymbol{x_1} + \boldsymbol{C_s} * \boldsymbol{x_3} - \boldsymbol{K_t} * \boldsymbol{x_2} + \boldsymbol{K_t} * \boldsymbol{u} = \boldsymbol{0} \\ \dot{\boldsymbol{x}}_{\boldsymbol{1}} = \boldsymbol{x_3} \\ \dot{\boldsymbol{x}}_{\boldsymbol{2}} = \boldsymbol{x_4} \\ \boldsymbol{x(0)} \end{cases} \quad \textbf{(1.4)}$$

Isolating the evolution of the state variables the following state-space representation comes out:

$$\begin{bmatrix} \dot{x}_1(t) \\ \dot{x}_2(t) \\ \dot{x}_3(t) \\ \dot{x}_4(t) \end{bmatrix} = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ -\frac{K_S}{M} & \frac{K_S}{M} & -\frac{C_S}{M} & \frac{C_S}{M} \\ \frac{K_S}{m} & -\frac{(K_S+K_t)}{m} & \frac{C_S}{m} & -\frac{C_S}{m} \end{bmatrix} * \begin{bmatrix} x_1(t) \\ x_2(t) \\ x_3(t) \\ x_4(t) \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ 0 \\ \frac{K_t}{m} \end{bmatrix} * u(t) \quad \textbf{(1.5)}$$

$$y(t) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} * \begin{bmatrix} x_1(t) \\ x_2(t) \\ x_3(t) \\ x_4(t) \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \end{bmatrix} * u(t)$$

It is clear that the sensors measurements regard first and third state component (displacement and velocity of the sprung mass).

Once the system model is available, it's possible to verify some properties such as the stability.

To check if the system is stable let's have a look of the eigenvalues of matrix A:

$$\lambda_1 = -20.1478 + 78.5215\text{i}$$

$$\lambda_2 = -20.1478 - 78.5215\text{i}$$

$$\lambda_3 = -1.8522 + 5.2663\text{i}$$

$$\lambda_4 = -1.8522 - 5.2663\text{i}$$

Since they have $Re(\lambda_i) < 0 \quad i = 1, \dots, 4$ we can assert that the system is asymptotically stable. This is coherent to the fact that it is a description of the dynamic of a vehicle.

The matlab file "Quarter_Car_model" is the reference for the simulation. The following code is responsible for storing the parameters and for creating the continuous time system:

The system without presence of noises is characterized by the following parameters:

```
Ks=8000; %[N/m] spring stiffness
Cs=1000; %[N/m*s] damper stiffness
Kt=160000; %[N/m] tyre stifness
M=250; %[kg] sprung mass
m=25; %[kg] unsprung mass
A=[0 0 1 0;
   0 0 0 1;
   -Ks/M Ks/M -Cs/M Cs/M;
   Ks/m -(Ks+Kt)/m Cs/m -Cs/m];
B=[0;0;0;Kt/m];
C=[1 0 0 0;
   0 0 1 0];
nx=length(A); %number of states
sizeY=size(C);
ny=sizeY(1); %number of output
sizeU=size(B);
nu=sizeU(2); %number of input
D=zeros(ny,nu);
system_tc=ss(A,B,C,D);
```

In order to apply the Kalman theory it's necessary to discretize, hence to set a sampling time T_s. For this choice the following rule of thumb has been respected:

$$T_s \leq \frac{2\pi}{20\omega_{bw}}$$

With ω_{bw} bandwith pulsation equal to the absolute value of the dominant pole of the matrix A.

```
Ts=0.01;
system_td=c2d(system_tc,Ts);
[Ad,Bd,Cd,Dd]=ssdata(system_td);
```

By looking at the eigenvalues of the dynamic matrix Ad we can check again the stability:

$$\lambda_{1d} = 0.5782 + 0.5780$$

$$\lambda_{2d} = 0.5782 - 0.5780\text{i}$$

$$\lambda_{3d} = 0.9803 + 0.0517\text{i}$$

$$\lambda_{4d} = 0.9803 - 0.0517\text{i}$$

Indeed $|\lambda_i| < 1 \; for \; i = 1, \dots, 4$ suggests the asymptotic stability.

2. KALMAN FILTERING

As introduced previously the aim of this project is to design a Kalman Filter for the system ***(1.5)*** improving its performance by the RTS algorithm. Kalman theory finds a huge field of applications in several sectors since it allows to estimate the value of a variable in a noisy framework. It has been already introduced that most of the time noisy signals arise due to the conditions in which the system is working in or, basically, due to inaccuracy of the sensors used. Thus, considering the value of the state as it is described by the devices would be a bad strategy for designing a control law based, for example, on state feedback. Moreover, it is not always possible to have a direct access to these measurements. In these cases, the state remains "confined" as an internal variable whose value can be estimated through a filtering procedure on the output that by definition is a known signal of the system. The Kalman filter is based on Bayesian theory which provides an approach to estimate the value of a random variable from the value of another random variable. In the context of interest, the unknown variable is the state while the known one is the output. Both of them are random since they are affected respectively by the process and measurement noise. The most important result of the Bayesian theory is the following:

Bayes Estimator

Given a random variable $\theta \in R^n$ and a set of observations each one defined with a vector $d \in R^p$:

$$\theta = \begin{bmatrix} \theta_1 \\ \theta_2 \\ \vdots \\ \theta_n \end{bmatrix} \qquad d_k = \begin{bmatrix} \mathrm{d}_{1k} \\ d_2 k \\ \vdots \\ \mathrm{d}_{pk} \end{bmatrix}$$

Where:

- $\theta_i \;\; i = 1, \dots, n$ is the i-th component of θ
- $\mathrm{d}_{jk} \;\; j = 1, \dots, p$ is the j-th components of the observation d performed at time k.

By supposing m measurements, the best estimator $\hat{\theta}$ capable to minimize the quantity $E[(\hat{\theta} - \theta)^2]$ is:

$$\hat{\theta} = \Lambda_{\theta \mathrm{d}} * \Lambda_{dd}^{-1} * d$$

With:

- $\Lambda_{\theta \mathrm{d}} \in R^{nx(p*m)}$ covariance matrix between the vector parameters θ and the observations d:

$$\Lambda_{\theta \mathrm{d}} = \begin{bmatrix} Cov(\theta_1, \mathrm{d}_{11}) & \cdots & Cov(\theta_1, \mathrm{d}_{1m}) & Cov(\theta_1, \mathrm{d}_{21}) & \cdots & Cov(\theta_1, \mathrm{d}_{p1}) & \cdots & Cov(\theta_1, \mathrm{d}_{pm}) \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ Cov(\theta_n, \mathrm{d}_{11}) & \cdots & Cov(\theta_n, \mathrm{d}_{1m}) & Cov(\theta_n, \mathrm{d}_{21}) & \cdots & Cov(\theta_n, \mathrm{d}_{2m}) & \cdots & Cov(\theta_n, \mathrm{d}_{pm}) \end{bmatrix} \quad \textbf{(2.1)}$$

Where $\mathrm{d}_{ij}\ i = 1, \dots, p \quad j = 1, \dots, m$ is the j-th measurement of the i-th component of d and each single covariance is computed as follows:

$$E[(\theta_i - E[(\theta_i])(\mathrm{d}_{ij} - E[\mathrm{d}_{ij}])]$$

- $\Lambda_{\mathrm{dd}} \in R^{(p*m)x(p*m)}$ represents the covariance matrix of the observations d:

$$\Lambda_{\mathrm{dd}} = \begin{bmatrix} Var(\mathrm{d}_{11}) & Cov(\mathrm{d}_{11}, \mathrm{d}_{12}) & \cdots & Cov(\mathrm{d}_{11}, \mathrm{d}_{1m}) & Cov(\mathrm{d}_{11}, \mathrm{d}_{21}) & \cdots & Cov(\mathrm{d}_{11}, \mathrm{d}_{pm}) \\ Cov(\mathrm{d}_{12}, \mathrm{d}_{11}) & Var(\mathrm{d}_{12}) & \cdots & Cov(\mathrm{d}_{12}, \mathrm{d}_{1m}) & Cov(\mathrm{d}_{12}, \mathrm{d}_{21}) & \cdots & Cov(\mathrm{d}_{12}, \mathrm{d}_{pm}) \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ Cov(\mathrm{d}_{1m}, \mathrm{d}_{11}) & Cov(\mathrm{d}_{1m}, \mathrm{d}_{12}) & \cdots & Var(\mathrm{d}_{1m}) & Cov(\mathrm{d}_{1m}, \mathrm{d}_{21}) & \cdots & Cov(\mathrm{d}_{1m}, \mathrm{d}_{pm}) \\ Cov(\mathrm{d}_{21}, \mathrm{d}_{11}) & Cov(\mathrm{d}_{21}, \mathrm{d}_{12}) & \cdots & Cov(\mathrm{d}_{21}, \mathrm{d}_{1m}) & Var(\mathrm{d}_{21}) & \cdots & Cov(\mathrm{d}_{21}, \mathrm{d}_{pm}) \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ Cov(\mathrm{d}_{pm}, \mathrm{d}_{11}) & Cov(\mathrm{d}_{pm}, \mathrm{d}_{12}) & \cdots & Cov(\mathrm{d}_{pm}, \mathrm{d}_{1m}) & Cov(\mathrm{d}_{pm}, \mathrm{d}_{21}) & \cdots & Var(\mathrm{d}_{pm}) \end{bmatrix} \quad \textbf{(2.2)}$$

Where each variance is computed as follows:

$$Var(\mathrm{d}_{ij}) = E[(\mathrm{d}_{ij} - E[\mathrm{d}_{ij}])^2]$$

In the context of interest, the set of observations d is represented by the output measurements while the θ vector is the state.

The result described above is known as a batch formula because in order to design the estimator we need all the observations. There exists a recursive version of the Bayes estimator which is capable to update the estimation of the unknown variable every time a new incoming measurement is performed:

Recursive Bayes Estimator

Given a random variable $\theta \in R^n$ and a set of N observations each one characterized by a vector $d \in R^p$, the recursive bayes estimator is updated as follows:

$$\widehat{\boldsymbol{\theta}} = \boldsymbol{E}[\boldsymbol{\theta}|\boldsymbol{d_1}, \dots, \boldsymbol{d_{N-1}}] + \boldsymbol{E}[\boldsymbol{\theta}|\boldsymbol{e}] \quad \textbf{(2.3)}$$

Where:

- $[\theta|\mathrm{d}_1, \dots, \mathrm{d}_{N-1}]$ represents the estimation of θ based on the N-1 past observations.
- $E[\theta|\mathrm{e}]$ represents the estimation of θ based on the innovation term defined as the difference between the new measurement and the estimation of it considering the past observations:

$$e = \mathrm{d}_N - E[\mathrm{d}_N|\mathrm{d}_1, \dots, \mathrm{d}_{N-1}]$$

For instance, after having obtained the first measurement d_1, once the measurement $\mathrm{d}_2$is available the estimator is:

$$\hat{\theta} = E[\theta|\mathrm{d}_1] + E[\theta|\mathrm{e}]$$

With:

$$e = \mathrm{d}_2 - E[\mathrm{d}_2|\mathrm{d}_1] = \mathrm{d}_2 - \Lambda_{d_1 d_2} * \Lambda_{d_1 d_1}^{-1} * d_1$$

And:

$$E[\theta|\mathrm{e}] = \Lambda_{\theta \mathrm{e}} * \Lambda_{ee}^{-1} * e$$

This emphatises the 'real-time' nature of the algorithm.

Both fashions of the estimator are used to derive the Kalman Filter.

This filter is capable to perform both a prediction in the future and a current estimation of the state. Its evolution is described by the following equations:

$$\begin{cases} \boldsymbol{\hat{x}_{k|k} = \hat{x}_{k|k-1} + K_k * (y_k - \hat{y}_{k|k-1})} \\ \boldsymbol{P_{K|k} = P_{K|k-1} - K_k * C * P_{K|k-1}} \\ \boldsymbol{\hat{y}_{k|k-1} = C * \hat{x}_{k|k-1}} \\ \boldsymbol{K_k = P_{K|k-1} * C^T (C * P_{K|k-1} * C^T + V_2)^{-1}} \\ \boldsymbol{\hat{x}_{k+1|k} = A * \hat{x}_{k|k} + B * u_k} \\ \boldsymbol{P_{K+1|k} = A * P_{K|k} * A^T + V_1} \\ \boldsymbol{k = k + 1} \end{cases} \quad \textbf{(2.4)}$$

Where the notation $'variable\_name_{i|i-1}'$ represents the value of the variable at time instant i given the information (hence, the measurements) up to time $i-1$. The matrix P is the covariance matrix of the state and gives information about the uncertainty of the estimation. An objective consists of minimizing it as much as possible in order to have better estimation.

Thus, Kalman Filter is an algorithm divided into two parts:

1. The first four equations represent the *estimation* of the state, covariance matrix P and output.
2. The last two equations represent the *prediction* of state and matrix P.

It is possible to notice that the algorithm proceeds combining these two parts. The starting point is the initial condition of the state and of the covariance matrix P. The initial state can be associated considering an a-priori knowledge on it and as a consequence the covariance matrix P can be weighted taking into account how much this knowledge is reliable.

Moreover, the matrix $P(k)$ represents the solution of the Difference Riccati Equation (DRE) at time k:

$$\boldsymbol{P(k+1) = A * P(k) * A^T + V_1 - K(k) * C * P(k) * A^T} \quad \textbf{(2.6)}$$

If the algorithm converges, the matrix $P(k)$ and the gain $K(k)$ converge to constant values $\overline{P}$ and $\overline{K}$ ensuring stability of the filter. In this case P becomes the unique solution of the so called Algebraic Riccati Equation (ARE):

$$\boldsymbol{\overline{P} = A * \overline{P} * A^T + V_1 - \overline{K} * C * \overline{P} * A^T} \quad \textbf{(2.7)}$$

The convergence conditions are related to the following two properties:

1. (A, C) detectable
2. $(A, \sqrt{V_1})$ stabilizable, where the operation $\sqrt{V_1}$ is such that: $\sqrt{V_1} * (\sqrt{V_1})^T = V_1$

To check these features, let's proceed as follows:

```
>> rank(obsv(Ad,Cd))

ans =

     4
```

```
>> rank(ctrb(Ad,sqrtm(cov(v1))))

ans =

     4
```

Since the system has four states and it is asymptotically stable, it's possible to state that it is completely observable (so detectable) and the couple $(A, \sqrt{V_1})$ is stabilizable.

Once the convergence conditions are satisfied, we can move on designing the filter. First let's store the last parameters:

Let's define the noisy signals:

```
tsim=20; %simulation time [sec]
N=round((tsim/Ts)+1); %number of samples
v1=randn(N,nx)*sqrt(0.01); %0.1
t_v1=[0:Ts:tsim]';
process_noise=[t_v1,v1];
v2=randn(N,ny)*sqrt(30); %30
measurement_noise=[t_v1,v2];
```

Initial conditions:

```
x0=zeros(nx,1);
P0=10^-3*eye(nx);
```

Input parameters:

```
f=0.159; %[Hz] frequency of the bumps
as=8; %[cm] height of the strips
ab=30; %[cm] heigth of the bumps
astep=20; %[N] force amplitude of the step
input_choice=input('Choose the input signal: 1 for strips, 2 for bumps, 3 for step:
-> '); %selector value (1 for strips,2 for bumps,3 for step)
```

```
out=sim("Kalman_filtering.slx");
```

The process noise is characterized by a low variance since the evolution described in the ***chapter 1*** is based on physical analysis, thus it represents a reliable model. On the contrary, the measurement noise has high variance to put in evidence the filtering procedure we want to perform. Similar argument about the initial condition. At the beginning the car has no road input sources, therefore it is possible to state that both the displacement and the velocity of the

sprung mass are equal to zero. Based on this knowledge, the respective covariance matrix is characterized by small values. Input parameters refer to the possibility to choose by the selector variable "*input_choice*" among three different input signals:

1. A sequence of strips 8 [cm] high.
2. A sequence of two bumps 30 [cm] high.
3. A step of amplitude 20 [N].

At the end the filtering procedure is launched by the Simulink file "*Kalman_Filtering*". Here, a snapshot of it:

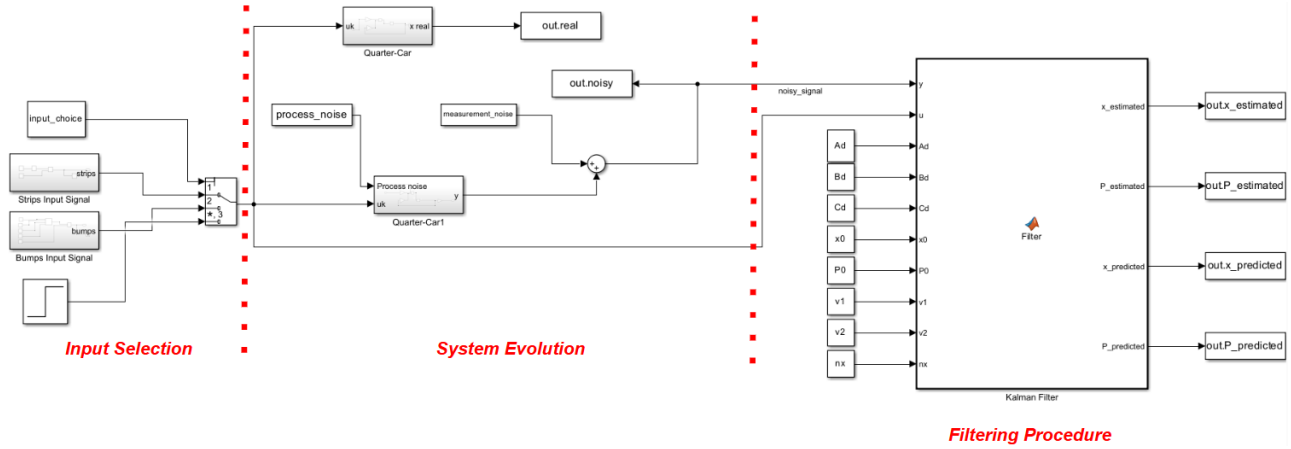

Fig. 2.1: Implementation and filtering of the system (Simulink)

The model is organized in three parts:

1. The *Input Selection* part conveys the signal chosen by the user to the system.
2. The System Evolution is responsible for implementing the evolution of the Quarter-Car model through the subsystem blocks "Quarter_Car". The upper block represents an ideal situation in which we are in charge of obtaining the displacement and velocity values of the sprung mass without presence of any kind of noises:

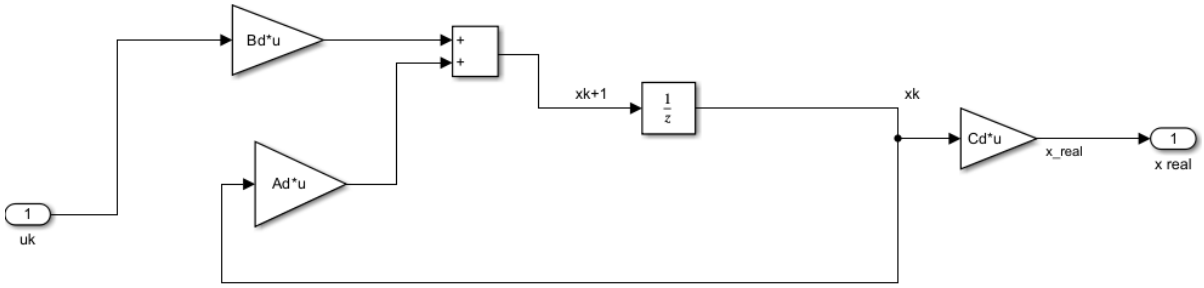

Fig. 2.2: Quarter-Car System without noises (Simulink)

So, they represent the real values (Cd is the extraction matrix). The other block models a real scenario providing just the output of the system (subjected to the process noise) which is after affected by the measurement noise:

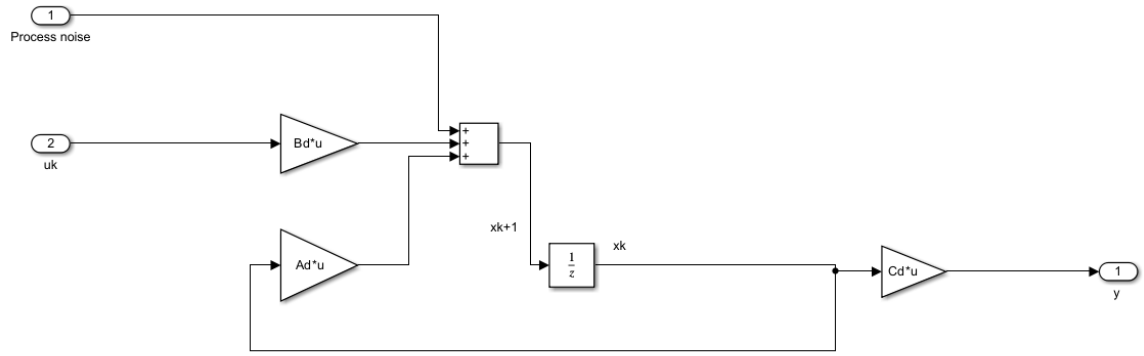

Fig. 2.3: Quarter-Car System with process noise (Simulink)

3. Filtering Procedure part receives the noisy measurement and implements the Kalman Filter through the Matlab function "*Filter*" to obtain a good estimation of the states (other outputs will be useful for future smoothing operation):

```
function [x_estimated,P_estimated,x_predicted,P_predicted] = Filter(y,u,Ad,Bd,Cd,x0,P0,v1,v2,nx)
    persistent P_kk_k;
    persistent x_kk_k;

    if isempty(x_kk_k)
       x_estimated=x0;
       P_estimated=P0;
       x_predicted=Ad*x_estimated+Bd*u;
       P_predicted=Ad*P_estimated*Ad'+cov(v1);
       x_kk_k=x_predicted;
       P_kk_k=P_predicted;
       return;
    end
```

```
      % Estimation phase
        e=y-Cd*x_kk_k; %innovation term
        S=Cd*P_kk_k*Cd'+cov(v2); %innovation covariance
        kalman_gain=P_kk_k*Cd'*inv(S);
        x_estimated=x_kk_k+kalman_gain*e;
        P_estimated=(eye(nx)-kalman_gain*Cd)*P_kk_k;

      % Prediction phase
        x_kk_k=Ad*x_estimated+Bd*u;
        P_kk_k=Ad*P_estimated*Ad'+cov(v1);
        x_predicted=x_kk_k;
        P_predicted=P_kk_k;
end
```

The algorithm is implemented using two persistent variables representing the last prediction computed. The initialization is performed assigning to the first estimations the initial values of state and covariance matrix P (the only information available at the beginning). After, the equations of the filter are implemented. Notice that at each iteration the code provides both the current estimations $\hat{\mathbf{x}}_{\mathbf{k|k}}$, $P_{\mathbf{k|k}}$ (by the variables '*x_estimated*','*P_estimated*') and the one-step ahead predictions $\hat{\boldsymbol{x}}_{\boldsymbol{k+1|k}}$, $P_{\boldsymbol{k+1|k}}$ (by the variables '*x_predicted*', '*P_predicted*').

Now it is possible to have a look at the filtering results considering different inputs for the Quarte-Car model. The outcomes are computed by the Simulink File "*Plotting_results*"

2.1 STRIPS INPUT SIGNAL

This case analyses a sequence of strips 8 [cm] high acting on the vehicle with frequency 1 [Hz]:

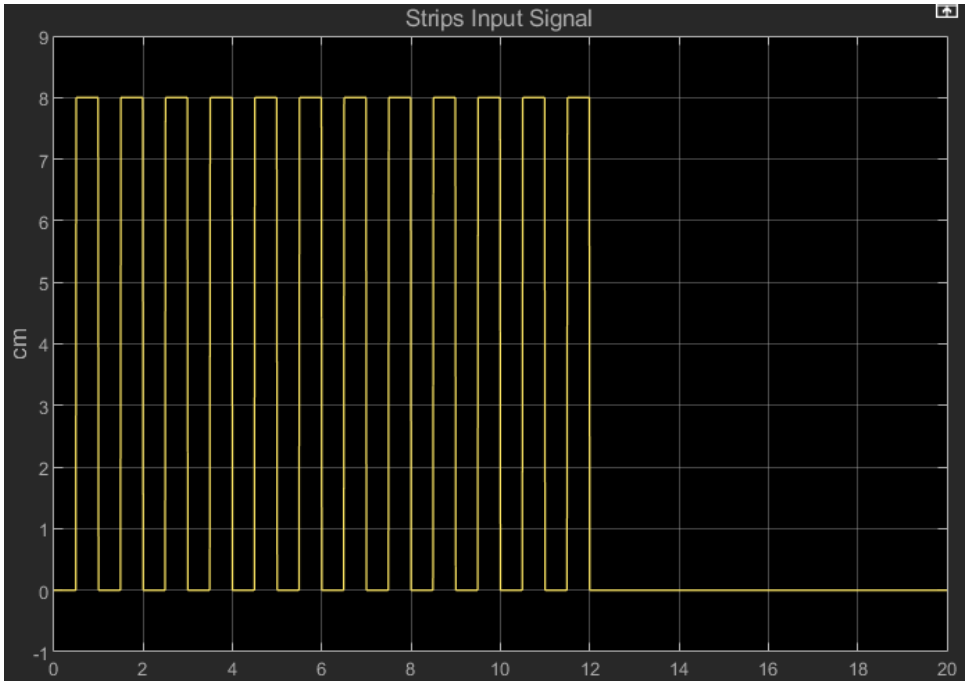

Fig. 2.4: Strips Input Signal (Simulink)

2.1.1 Displacement of the Sprung mass

The real response (state) of the system is:

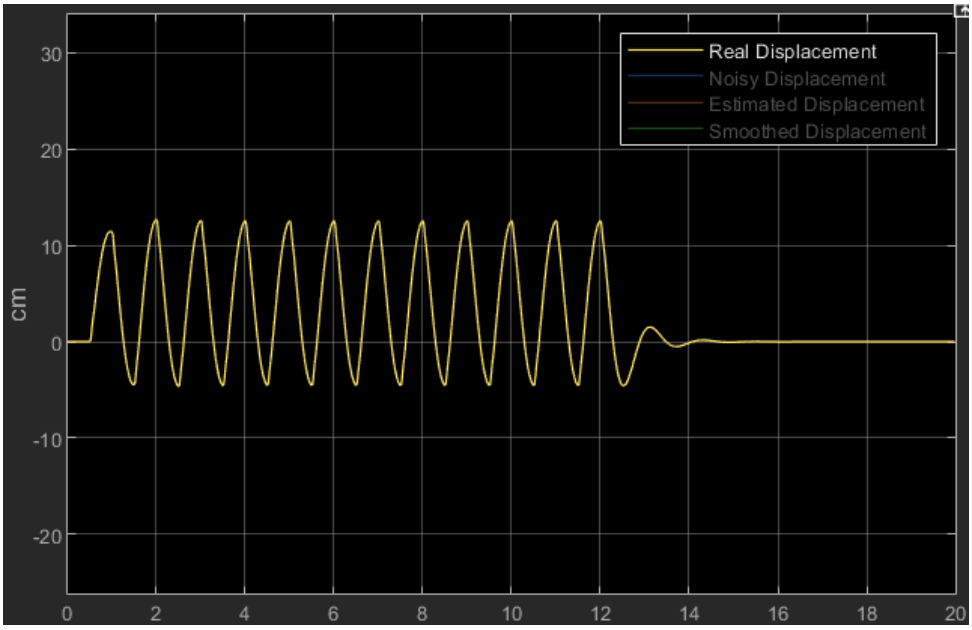

Fig. 2.5: Real Displacement of the Sprung Mass (Simulink)

The response of the system in presence of noises becomes:

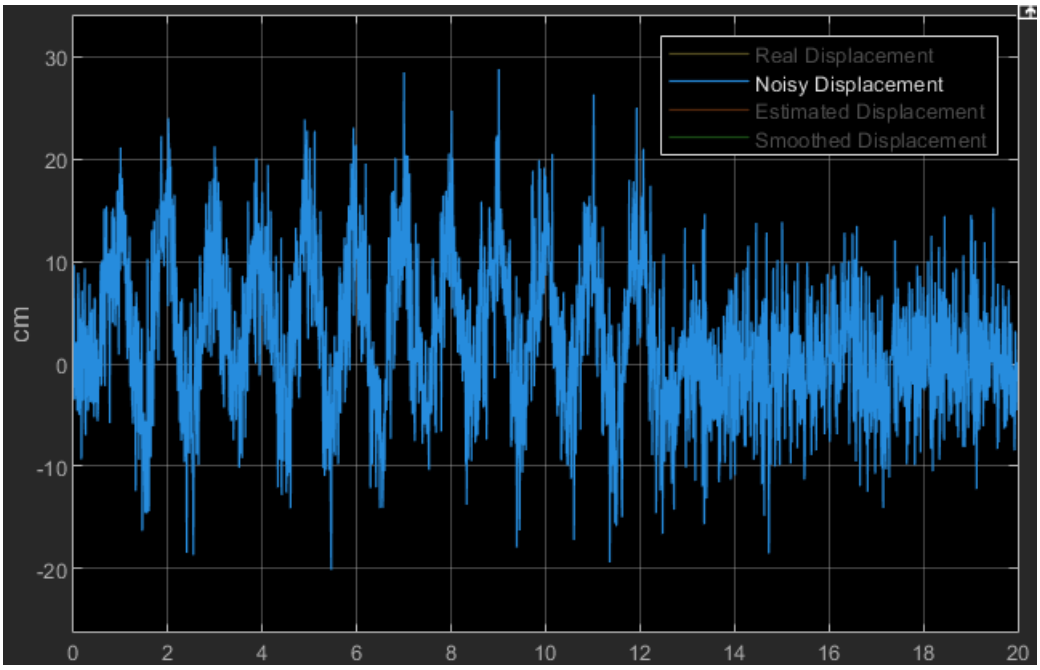

Fig. 2.6: Noisy Displacement of the Sprung Mass (Simulink)

The signal processed by the Kalman Filter compared with the real one is:

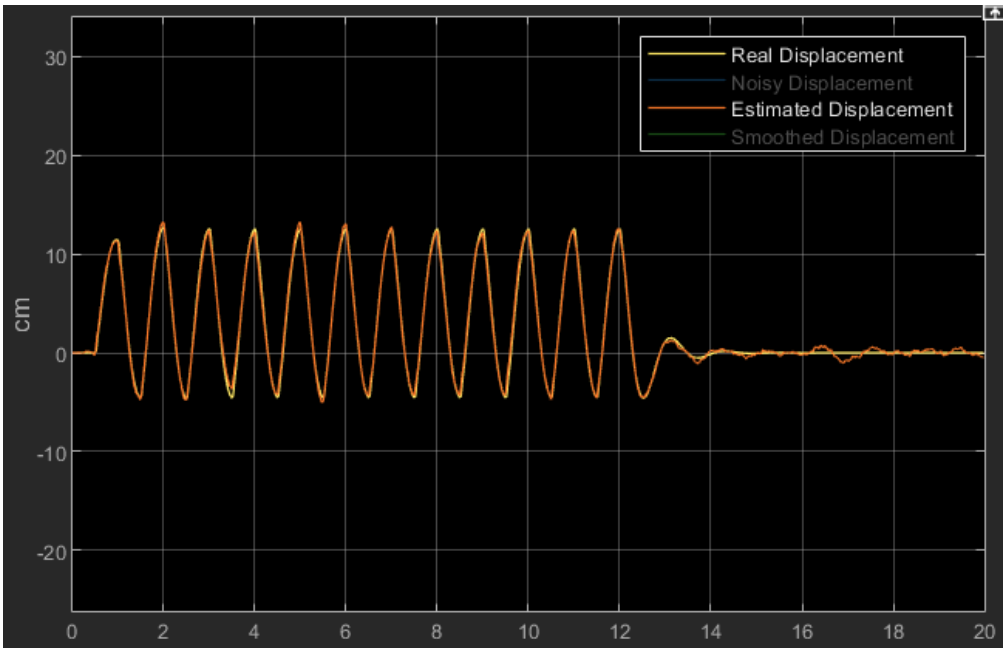

Fig. 2.7: Comparison between Real and Estimated Displacement of the Sprung Mass (Simulink)

2.1.2 Velocity of the Sprung mass

The output signal of the system without presence of noises is:

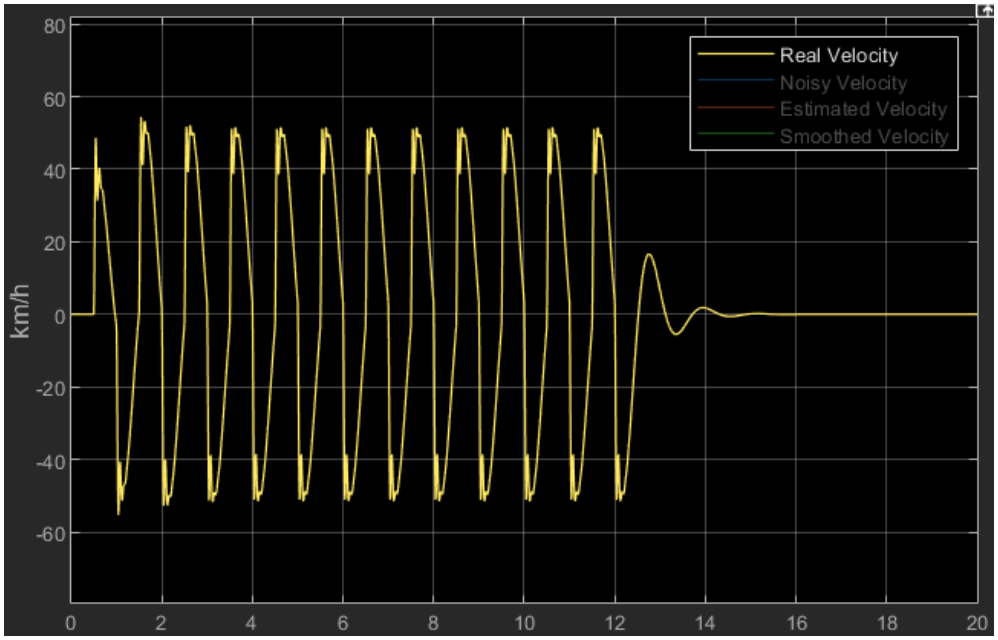

Fig. 2.8: Real Velocity of the Sprung Mass (Simulink)

When the noises arise:

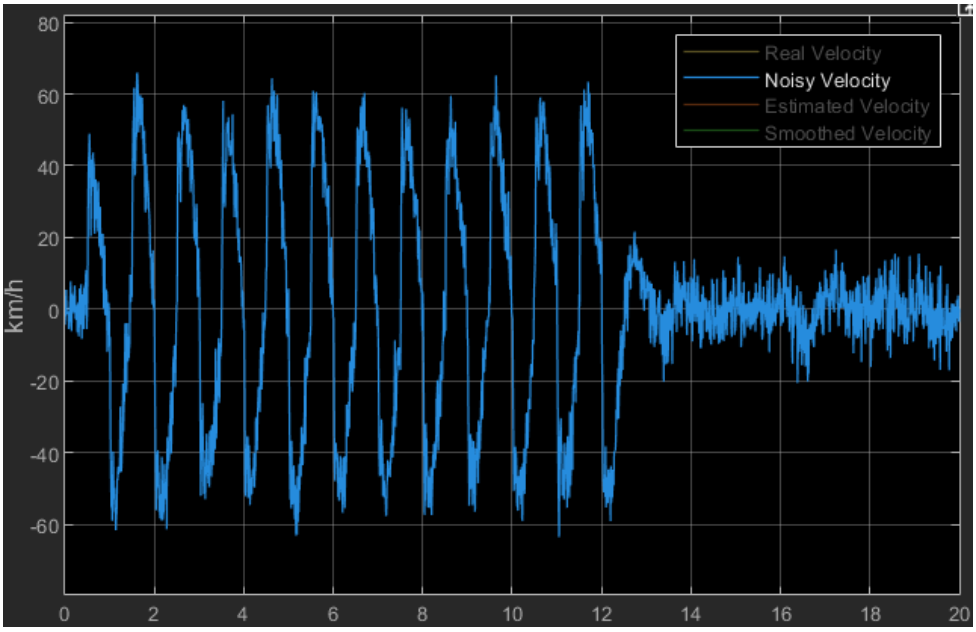

Fig. 2.9: Noisy Velocity of the Sprung Mass (Simulink)

The comparison between filtered and real signal is:

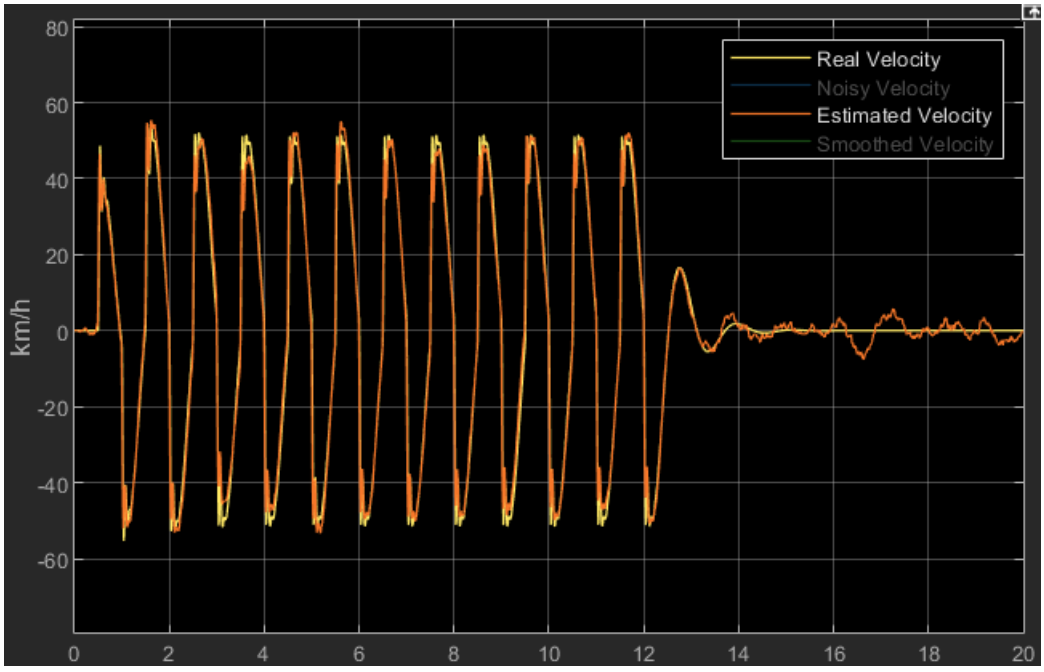

Fig. 2.10: Comparison between Real and Estimated Velocity of the Sprung Mass (Simulink)

It's possible to appreciate the action of the Kalman Filter for both the states.

Here a brief report of the results for the last two inputs:

2.2 BUMPS INPUT SIGNAL

This scenario describes a sequence of two bumps 30 [cm] high:

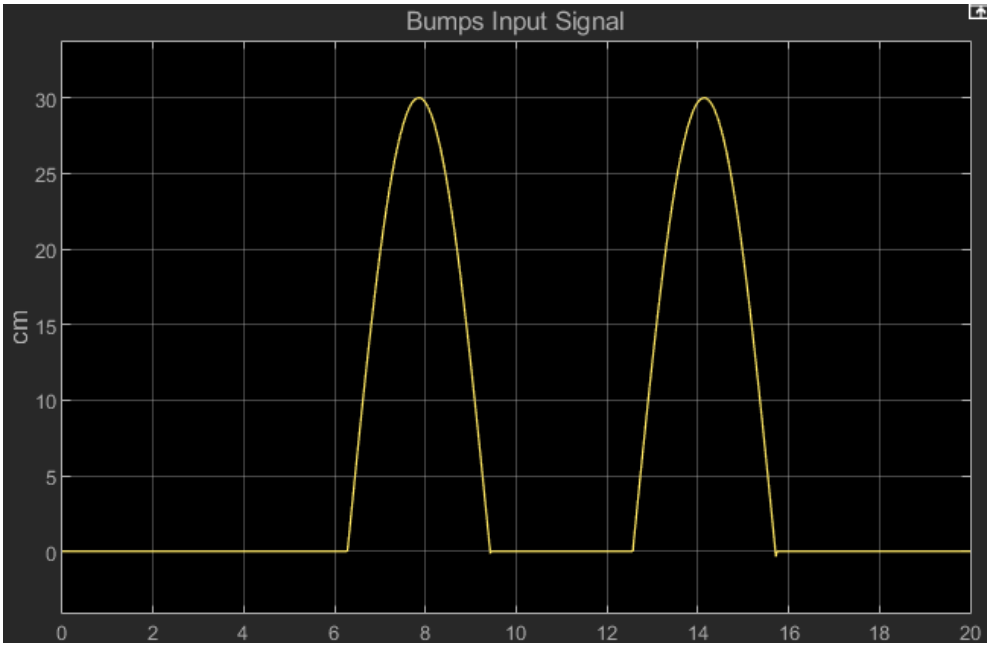

Fig. 2.11: Bumps Input Signal

2.2.1 Displacement of the Sprung mass

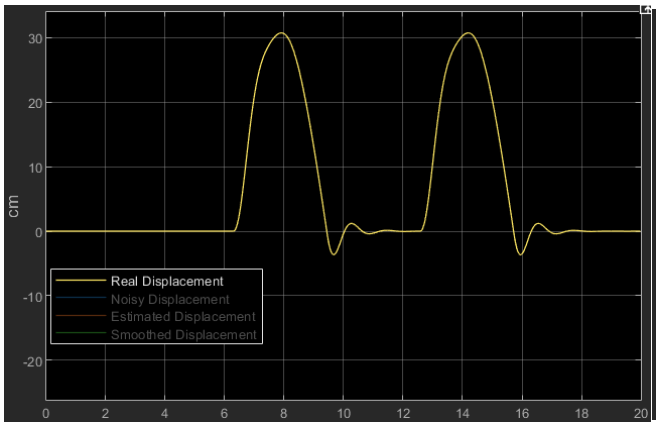

Fig. 2.12: Real Displacement of the Sprung Mass (Simulink)

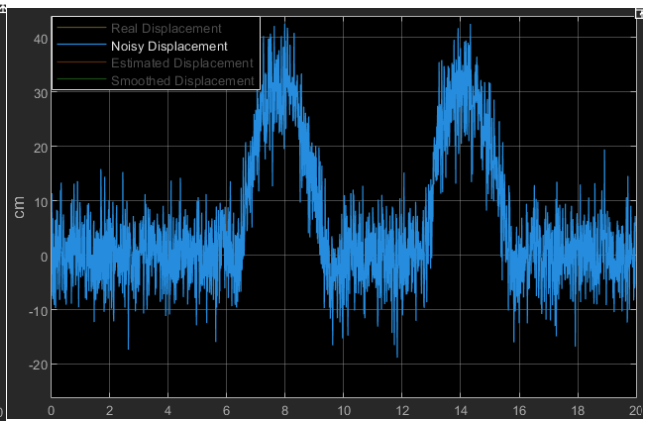

Fig. 2.13: Noisy Displacement of the Sprung Mass (Simulink)

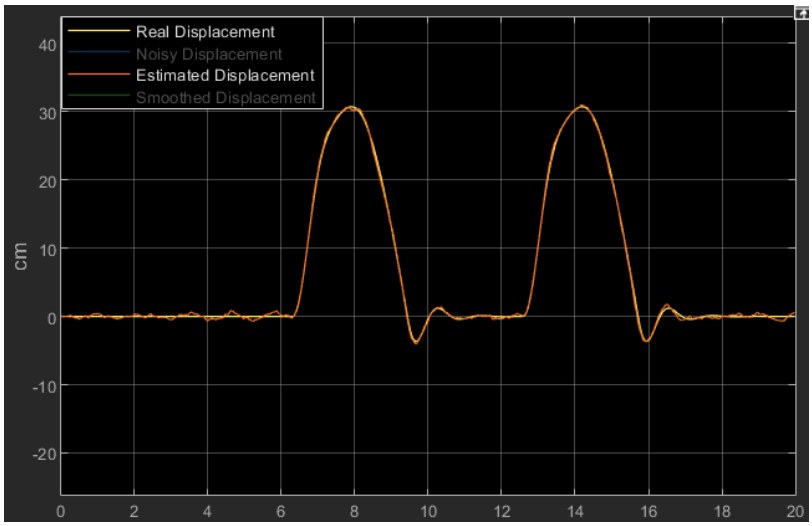

Fig. 2.14: Comparison between Real and Estimated Displacement of the Sprung Mass (Simulink)

2.2.2 Velocity of the Sprung mass

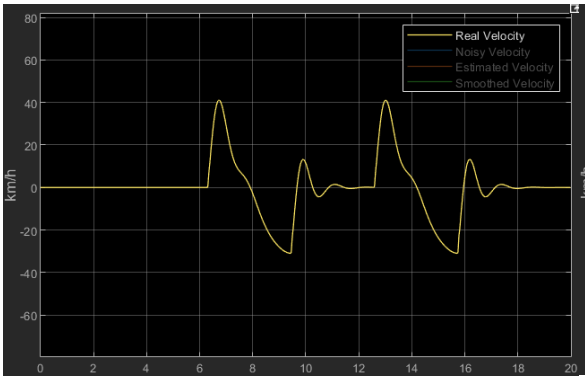

Fig. 2.15: Real Velocity of the Sprung Mass (Simulink)

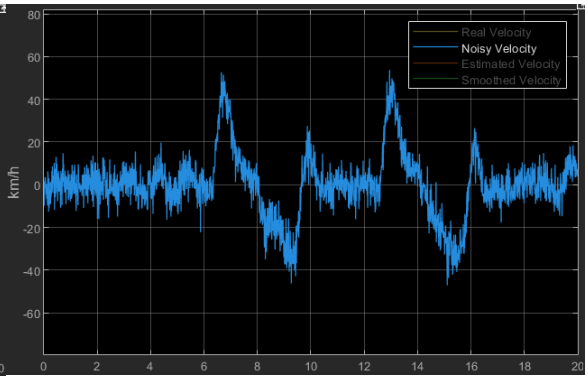

Fig. 2.16: Noisy Velocity of the Sprung Mass (Simulink)

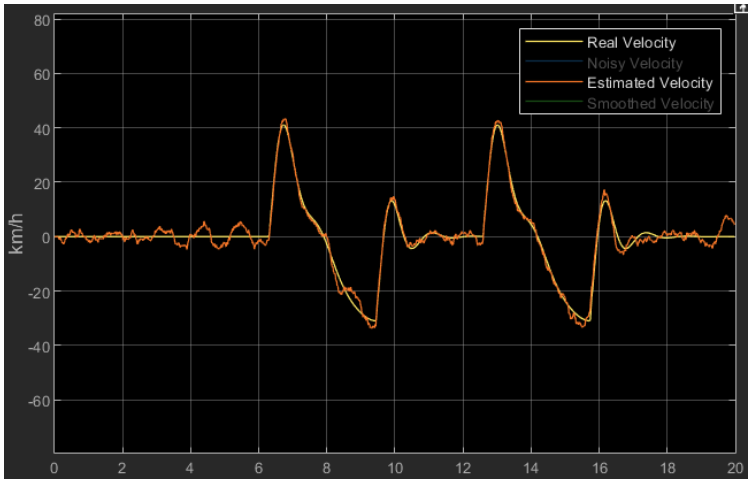

Fig. 2.17: Comparison between Real and Estimated Velocity of the Sprung Mass (Simulink)

2.3 STEP INPUT SIGNAL

Last scenario regards a common signal that is the step. In this context it should represents a force of constant intensity applied to the system. Even if it could not be considered as an appropriated road source, the respective results are reported:

2.3.1 Displacement of the Sprung mass

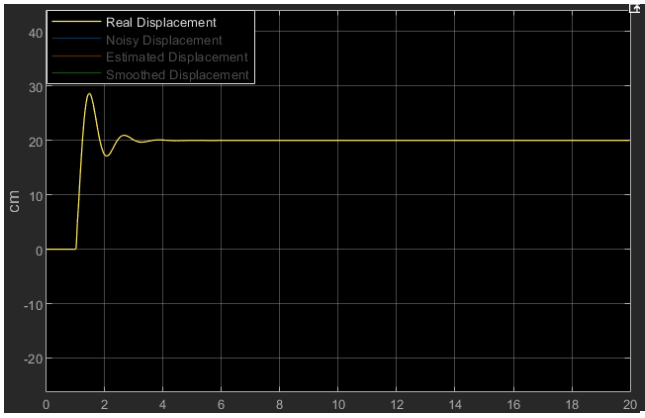

Fig. 2.18: Real Displacement of the Sprung Mass (Simulink)

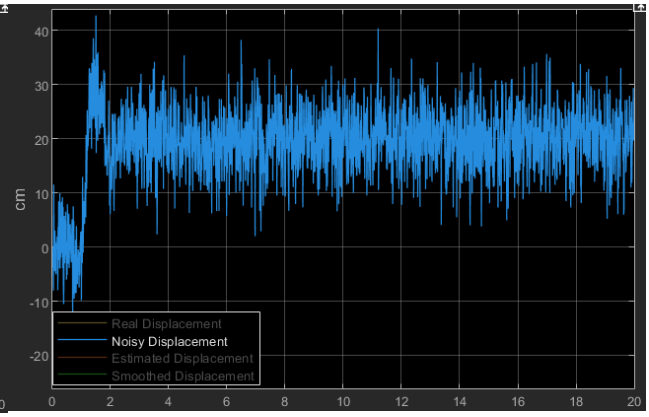

Fig. 2.19: Noisy Displacement of the Sprung Mass (Simulink)

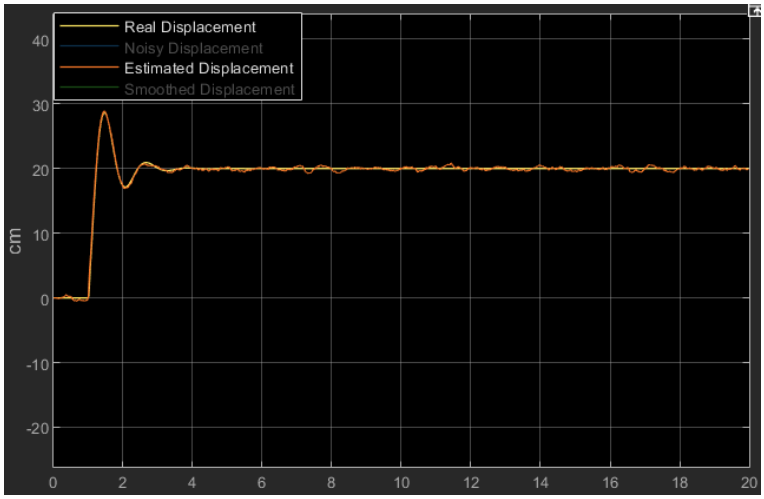

Fig. 2.20: Comparison between Real and Estimated Displacement of the Sprung Mass (Simulink)

2.3.2 Velocity of the Sprung mass

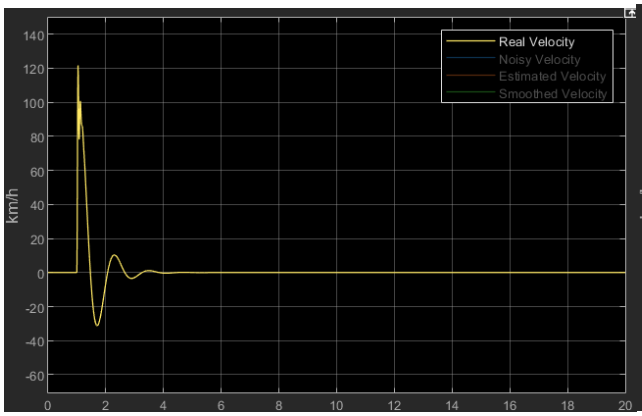

Fig. 2.21: Real Velocity of the Sprung Mass (Simulink)

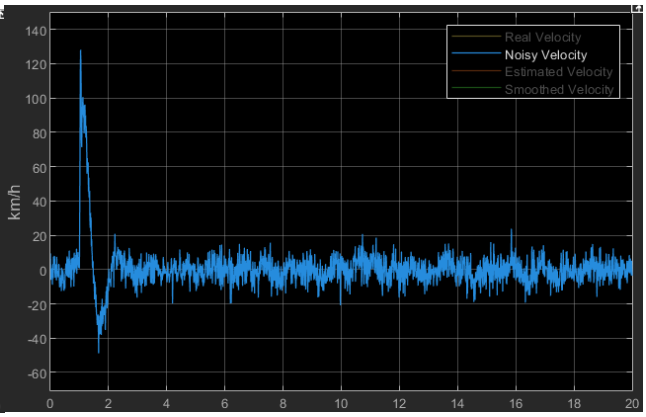

Fig. 2.22: Noisy Velocity of the Sprung Mass (Simulink)

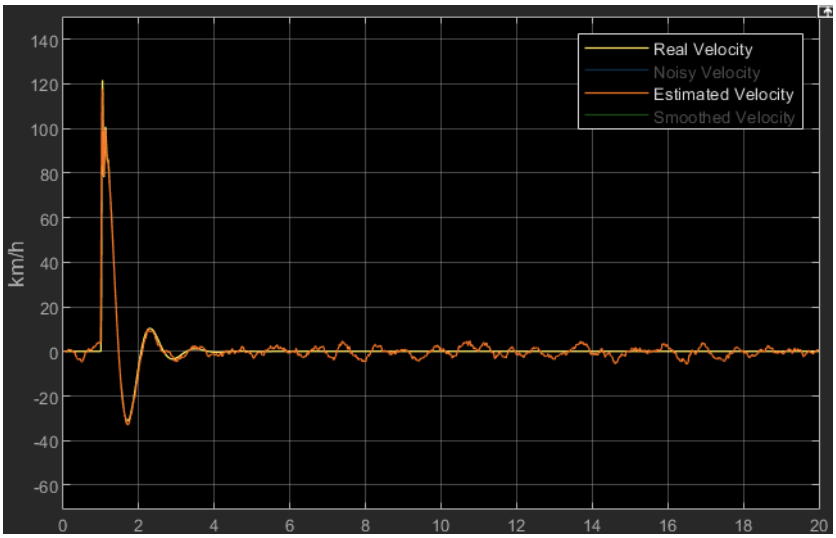

Fig. 2.23: Comparison between Real and Estimated Velocity of the Sprung Mass (Simulink)

3. RTS SMOOTHER

Once the Kalman Filter results are available, we want to investigate a way to improve the estimations provided. This kind of operation is said *smoothing* and to sort off the problem there exist different *smoothers*. A smoother is an algorithm that process data in a context of filtering procedure. More precisely, we want to focus on the RTS Smoother (Rauch, Tung, and Striebet). It belongs to a class of algorithms which work on a fixed time interval. This means that a time interval of observation must be selected and the procedure will smooth the values belonging to that. RTS smoother is based on two operations:

1. On-line phase: during this phase a Kalman Filter is used to store the estimations of the state and of the covariance matrix during the interval of observation. This part is related to a forward evolution, in the sense that the Kalman Filter processes data from the initial time instant to the last.
2. Off-line phase: once all the data are available, it's possible to start the smoothing operation. This phase is related to a backward evolution since the RTS smoother starts acting from the last estimation to the first. At each time instant the algorithm uses future information to improve the estimation computed by the Kalman Filter.

The smoother is characterized by the following equations:

$$\begin{cases} \boldsymbol{K_{RTS,k} = P_{Kalman,k} * A^T * \left(P_{k+1|k}\right)^{-1}} \\ \boldsymbol{x_{RTS,k} = x_{Kalman,k} + K_{RTS,k} * \left(x_{RTS,k+1} - x_{k+1|k}\right)} \\ \boldsymbol{P_{RTS,k} = P_{Kalman,k} + K_{RTS,k} * \left(P_{RTS,k+1} - P_{k+1|k}\right) * K_{RTS,k}^T} \\ \boldsymbol{x_{RTS,N} = x_{Kalman,N}} \\ \boldsymbol{P_{RTS,N} = P_{Kalman,N}} \\ \boldsymbol{k = k - 1} \end{cases} \qquad \textbf{(3.1)}$$

Where:

- The variables $x_{Kalman,k}$ and $P_{Kalman,k}$ are the estimated state and covariance at time k coming from the Kalman Filter.
- The variables $x_{k+1|k}$ and $P_{k+1|k}$ are the predited state and covariance at time k coming from the Kalman Filter.
- $K_{RTS,k}$ represents a gain for the RTS Smoother.
- The variables $x_{RTS,i}$ and $P_{RTS,i}$ are the smoothed state and covariance at time i.

Since the algorithm proceeds backward, the initial iteration k is set to N which represents the number of samples computed according to the Kalman procedure. Thus:

$$\begin{cases} x_{RTS,N} = x_{Kalman,N} \\ P_{RTS,N} = P_{Kalman,N} \\ k = N \end{cases}$$

Is the initialization of the smoother.

Now it's possible to apply this strategy to the Quarter-Car system. First, let's extract from Simulink the state estimated by the Kalman procedure:

Extract x1 and x3 estimated:

```
x1_estimated=out.x_estimated.Data(1,:,:);
x1_estimated=x1_estimated(:,:)';
x3_estimated=out.x_estimated.Data(3,:,:);
x3_estimated=x3_estimated(:,:)';
```

Previous operation is instrumental to initialize the smoother algorithm:

Smoothing Operation. Let's initialize the algorithm:

```
x_smoothed=zeros(length(out.tout),nx);
P_smoothed=zeros(nx,nx,length(out.tout));
x_smoothed(end,:)=out.x_estimated.Data(:,:,end)';
P_smoothed(:,:,end)=out.P_estimated.Data(:,:,end);
```

The variables '*x_smoothed*' and *P_smoothed*' will contain the improved estimations of the state and of the associated covariance matrix.

The following code lines refer to the implementation of the smoothing equations:

RTS smoother algorithm (Backward updating):

```
k=length(out.tout)-1;
while(k>0)
    K_RTS=out.P_estimated.Data(:,:,k)*Ad'*inv(out.P_predicted.Data(:,:,k));
    x_smoothed(k,:)=(out.x_estimated.Data(:,:,k)+K_RTS*(x_smoothed(k+1,:)'-
out.x_predicted.Data(:,:,k)))';
    P_smoothed(:,:,k)=out.P_estimated.Data(:,:,k)+K_RTS*(P_smoothed(:,:,k+1)-
out.P_predicted.Data(:,:,k))*K_RTS';
    k=k-1;
end
x1_smoothed=x_smoothed(:,1);
x3_smoothed=x_smoothed(:,3);
```

The evolution is implemented in the while loop. After the action of the smoother, the new state variables are extracted and stored in the variables '*x1_smoothed*' and '*x3_smoothed*'.

Let's have a look at the results. As it was shown previously, the outcomes are presented according to the input source:

3.1 STRIPS INPUT SIGNAL

3.1.1 Displacement of the Sprung mass

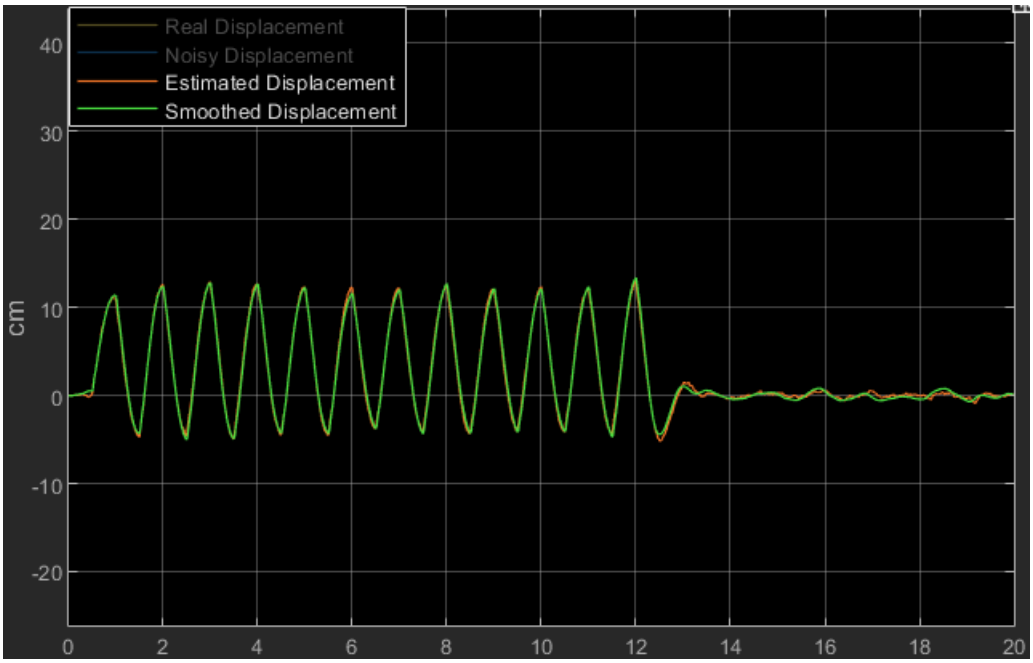

Fig. 3.1: Comparison between Smoothed and Estimated Displacement of the Sprung Mass (Simulink)

Despite the result of the smoother and of the Kalman could seem very similar in the time-trend, let's zoom the above snapshot in the interval [11 16] sec:

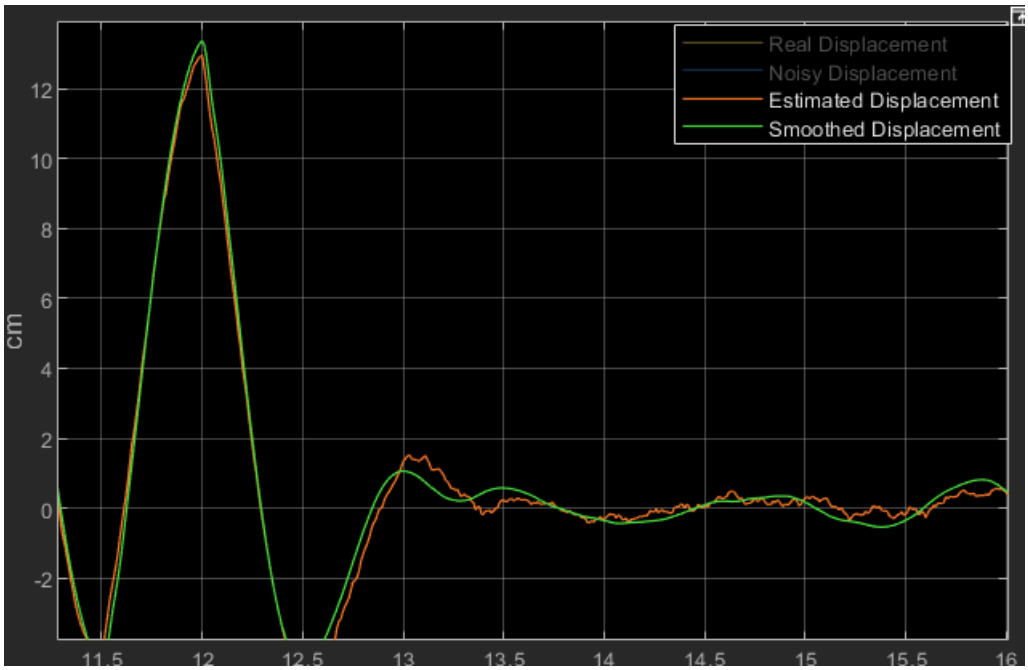

Fig. 3.1.1: Zoom of the Fig. 3.1

It's possible to appreciate that the green line (smoothed) behaves like a continuous signal rather than the red one which has much more discontinuities.

3.1.2 Velocity of the Sprung mass

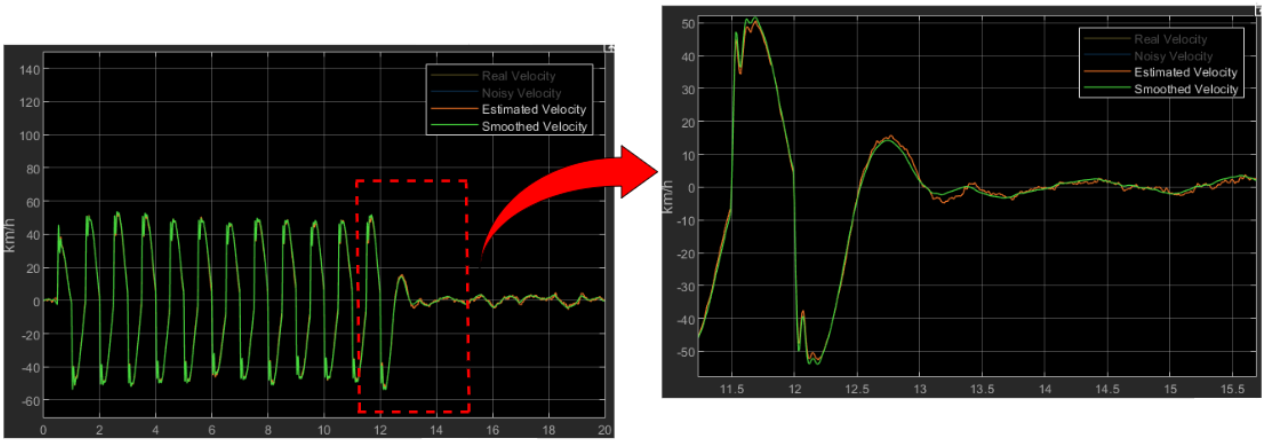

Fig. 3.2: Comparison between Smoothed and Estimated Velocity of the Sprung Mass (Simulink)

Here a brief report of the results for the last two inputs:

3.2 BUMPS INPUT SIGNAL

3.2.1 Displacement of the Sprung mass

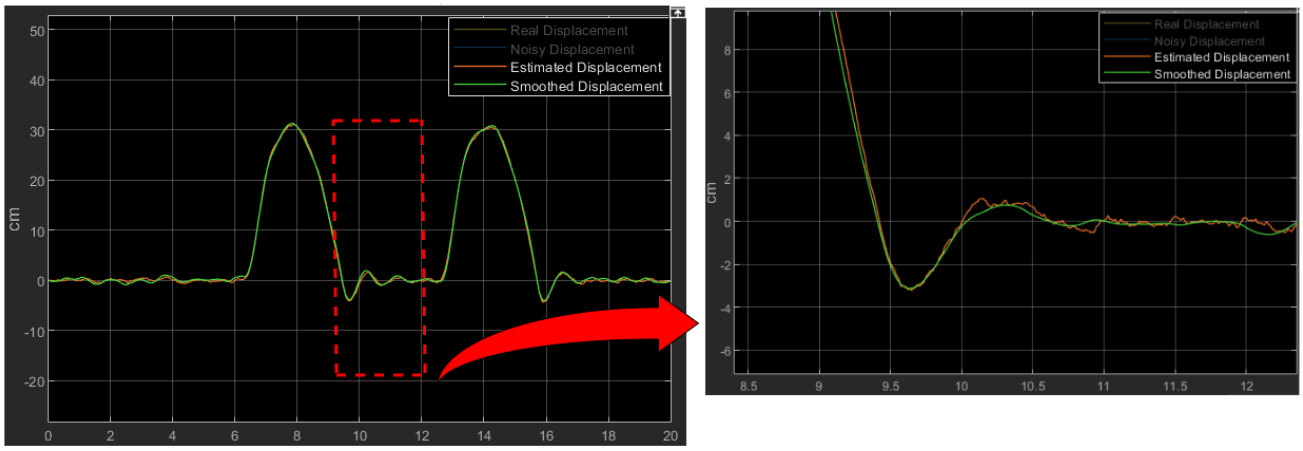

Fig. 3.3: Comparison between Smoothed and Estimated Displacement of the Sprung Mass (Simulink)

3.2.2 Velocity of the Sprung mass

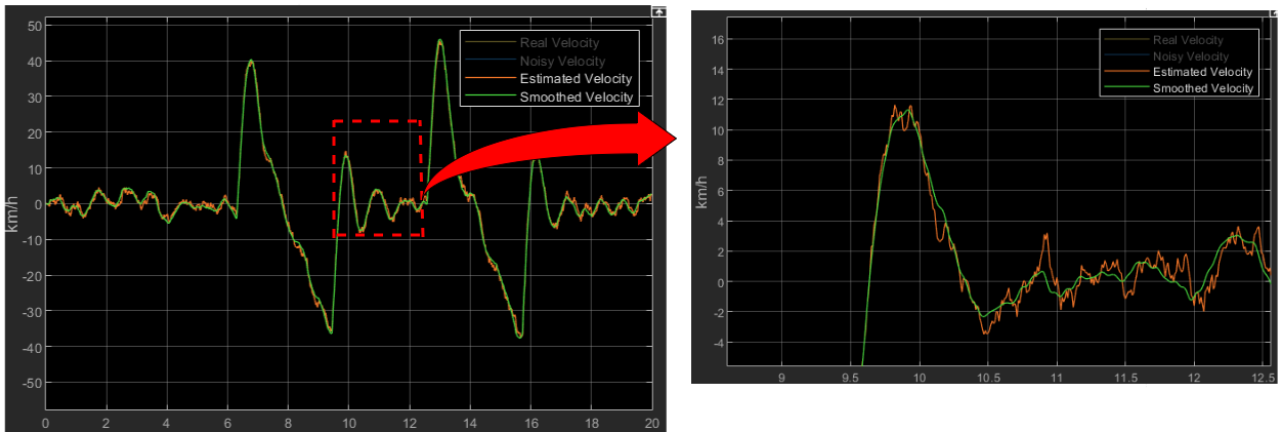

Fig. 3.4: Comparison between Smoothed and Estimated Velocity of the Sprung Mass (Simulink)

3.3 STEP INPUT SIGNAL

3.3.1 Displacement of the Sprung mass

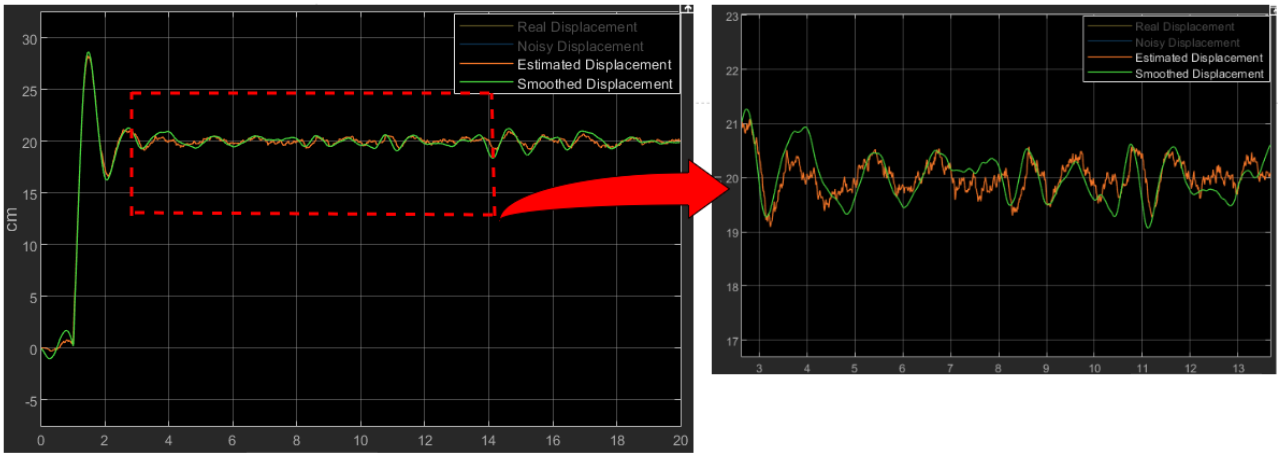

Fig. 3.5: Comparison between Smoothed and Estimated Displacement of the Sprung Mass (Simulink)

3.3.2 Velocity of the Sprung mass

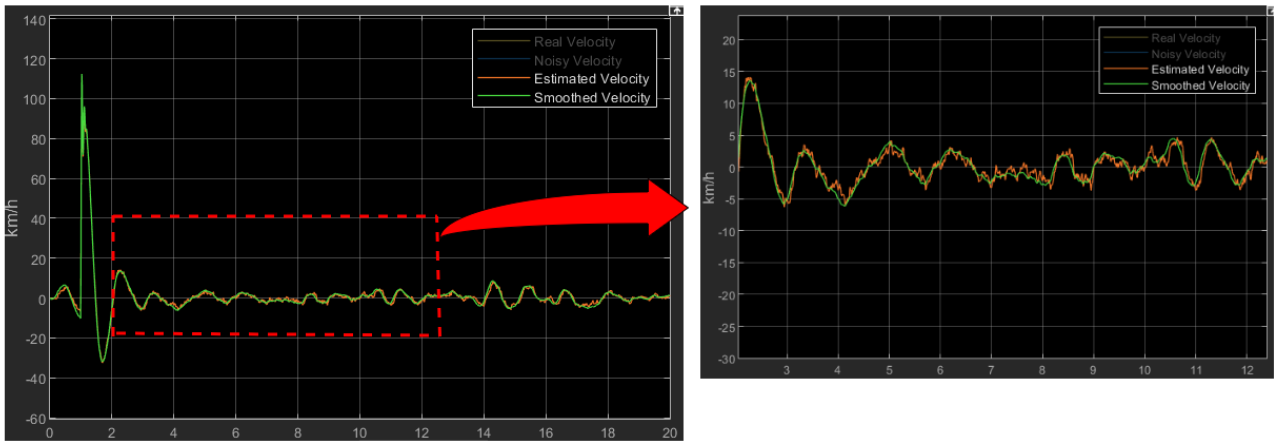

Fig. 3.6: Comparison between Smoothed and Estimated Velocity of the Sprung Mass (Simulink)

3.4 Loss of Transmission Scenario

To complete this work let's address a particular, but realistic, scenario. In this paragraph the aim consists of showing the behaviour of both the simple Kalman Filter and RTS Smoother when a loss of the communication (measurements and input) occurs during a time interval. Losses of transmissions are common situations due to several reasons such as network failure, environmental conditions, cyber-attack. In this context, it is fundamental to understand how much the Kalman Filter's estimations are reliable and if the RTS Smoother is still capable to improve them. To do this, let's modify the Matlab function in the Simulink file (*Fig.2.1*) to simulate the loss of packages:

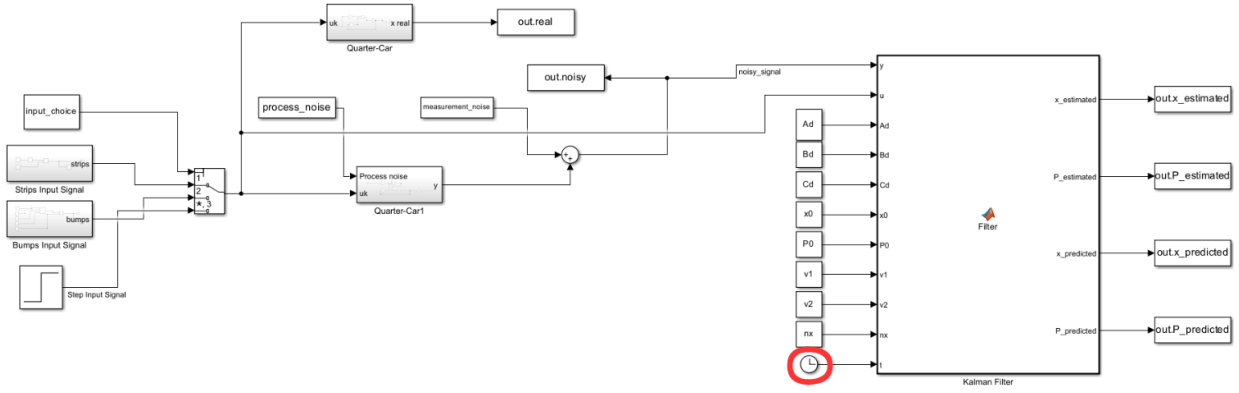

Fig. 3.7: New scenario implementation (Simulink)

The new scheme is implemented in the Simulink file "***Loss_of_transmission***". Notice that the Kalman Filter function now receives as input also the clock of the simulation in order to determine if losses occurred or not. Let's go deeper inside the function:

```
function [x_estimated,P_estimated,x_predicted,P_predicted] = Filter(y,u,Ad,Bd,Cd,x0,P0,v1,v2,nx,t)
    persistent P_kk_k;
    persistent x_kk_k;

    if isempty(x_kk_k)
       x_estimated=x0;
       P_estimated=P0;
       x_predicted=Ad*x_estimated+Bd*u;
       P_predicted=Ad*P_estimated*Ad'+cov(v1);
       x_kk_k=x_predicted;
       P_kk_k=P_predicted;
       return;
    end
```

```
  %Interruption of the measurements and input
  if (t>=7 && t<=14)
      % Estimation phase
      x_estimated=x_kk_k;
      P_estimated=P_kk_k;
      % Prediction phase
      x_kk_k=Ad*x_estimated;
      P_kk_k=Ad*P_estimated*Ad'+cov(v1);
      x_predicted=x_kk_k;
      P_predicted=P_kk_k;
```

```
   else
       % Estimation phase
       e=y-Cd*x_kk_k; %innovation term
       S=Cd*P_kk_k*Cd'+cov(v2); %innovation covariance
       kalman_gain=P_kk_k*Cd'*inv(S);
       x_estimated=x_kk_k+kalman_gain*e;
       P_estimated=(eye(nx)-kalman_gain*Cd)*P_kk_k;

       % Prediction phase
       x_kk_k=Ad*x_estimated+Bd*u;
       P_kk_k=Ad*P_estimated*Ad'+cov(v1);
       x_predicted=x_kk_k;
       P_predicted=P_kk_k;
   end
end
```

The evolution is incorporated into an *if/else* block to recognize if the current time instant belongs to the time interval [***7 14***] [*sec*], set as the period during which losses happen ([***7 10***] [*sec*] for strips signal). For what concerns the else condition, the code inside this block is perfectly equal to the case with all the data available. On the contrary, the if block describes the behaviour of the filter during the critical phase. Having a look at the equations ***2.4***, it is possible to state that in absence of both observations and input neither the Kalman gain and the forced response can be computed. Thus, the estimation is based only on the prediction and the prediction translates just into the free response of the system. In other words, the filter is still capable to work until new data are provided, after that it starts again its standard estimation procedure.

Now let's have a look at the results in terms of displacement of the sprung mass:

3.4.1 STRIPS INPUT SIGNAL (losses in [7 10] [sec])

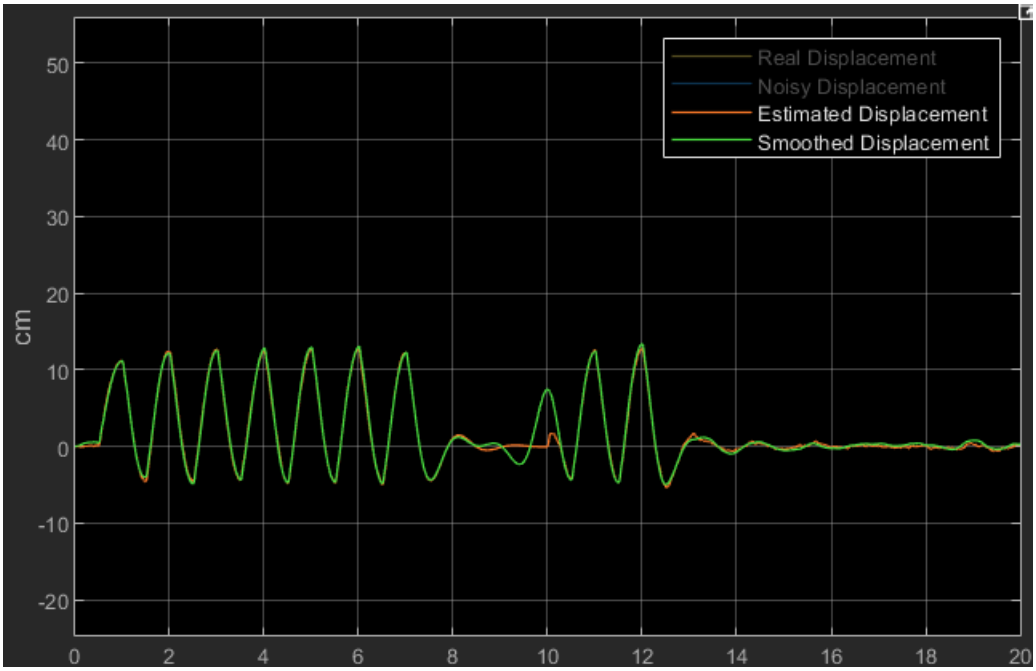

Fig. 3.8: Comparison between Estimated and Smoothed Displacement of the sprung mass (Simulink)

3.4.2 BUMPS INPUT SIGNAL

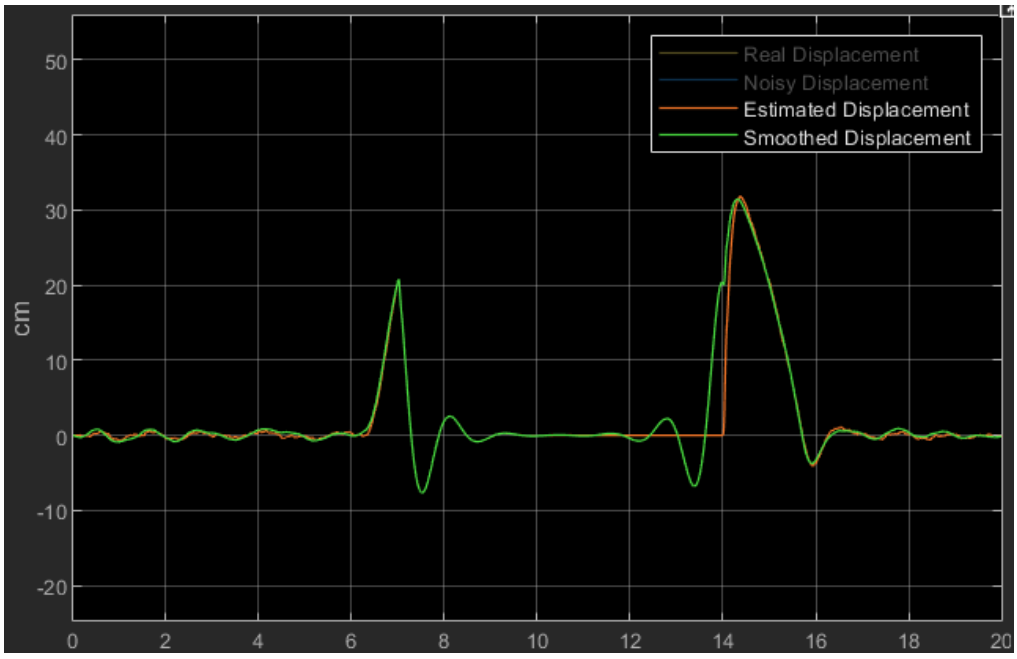

Fig. 3.9: Comparison between Estimated and Smoothed Displacement of the sprung mass (Simulink)

3.4.3 STEP INPUT SIGNAL (Step time 6 [sec])

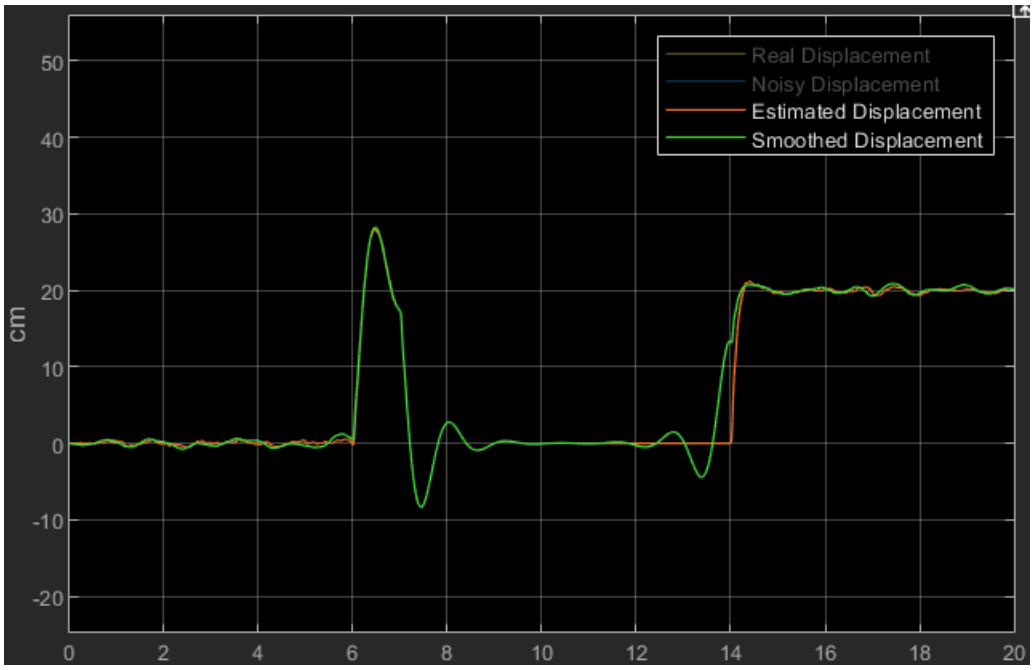

Fig. 3.10: Comparison between Estimated and Smoothed Displacement of the sprung mass (Simulink)

It's possible to notice that when losses occur the estimation (free response) goes to zero since the system is asymptotically stable. The filter restarts its standard procedure when new data are detected. Also in this case, the use of the RTS algorithm is useful. Indeed, it allows to have a smoother transition.